The Australian National University

School of Computing

JHQ-GPU

Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

Abstract

JHQ-GPU runs JHQ's primary/residual search on a GPU. Cartesian splitting replaces 256 LUT entries with $16 + 16$, giving a storage ratio $32/256 = 1/8$ with the same real-arithmetic score. Calibration compares sampled top-k IDs with a larger-budget reference, without GT or a learned predictor.

On six RTX 5090 datasets, two-group LUTs lead in 22/24 cells; the largest index holds 17.78 million vectors at 20.44 GiB resident memory. Full-pool calibration meets a 10^{-3} held-out recall-loss target relative to the best tested budget in 12/12 cells, with maximum loss 0.0006; smaller samples expose workload-dependent quality risk. At recall 0.97 and batch 1,000, JHQ leads in three of four datasets and is near parity with IVF-RaBitQ on arXiv. OpenAI runs use TF32; FP32 checks retain favourable comparisons under a TF32-trained cache. At batch 8 and the same recall, CAGRA fp32 leads on all four datasets. These results locate the execution, quality and capacity trade-offs of GPU JHQ.

Acknowledgements

I thank Dr Mengxuan Zhang for supervising this project and acknowledge the JHQ authors and open-source ANN/GPU software cited throughout this thesis.

Research and AI assistance. This thesis extends joint JHQ-GPU work with Mengxuan Zhang [21] and builds on JHQ [16]. OpenAI Codex assisted with substantive drafting, revision, LaTeX, analysis and code/log checks [30]; ChatGPT supplied frameworks and reviews [29]; Anthropic Claude assisted with implementation, experiments and review [4]. Selected prompts and usage details are available in the AI-use record. The author is responsible for the submitted claims.

Contents

Abstract i

Acknowledgements i

Notation and Abbreviations vi

1 Introduction 1
- 1.1 Context and Practical Objectives . . . 1
- 1.2 Problem Definition and Scope . . . 1
- 1.3 Three Linked Technical Challenges . . . 1
- 1.4 Research Questions and Falsifiable Claims . . . 2
- 1.5 Research Contributions . . . 2

2 Background and Related Work 4
- 2.1 ANN Objective and Metrics . . . 4
- 2.2 IVF, PQ, ADC and Refinement . . . 4
- 2.3 JHQ Representation and Assumptions . . . 5
- 2.4 GPU Execution Concepts . . . 6
- 2.5 Graph Search on GPUs . . . 6
- 2.6 Quantised Search and Fast Tables . . . 6
- 2.7 Research Gap and Design Choices . . . 6

3 Research Design and Measurement 8
- 3.1 Research Questions and Comparisons . . . 8
- 3.2 Datasets, Queries, Ground Truth and Splits . . . 8
- 3.3 Timing, Variance and Memory Accounting . . . 9
- 3.4 Baselines and Frontier Construction . . . 10
- 3.5 Experimental Records and Reproducibility . . . 11

4 GPU Index Design and Construction 12
- 4.1 End-to-End Build and Search Boundaries . . . 12
- 4.2 Training, Rotation and Codebook Generation . . . 12
- 4.3 IVF Assignment, IDs and Physical Memory Layout . . . 14
- 4.4 Resident Storage, Workspace and Index Lifecycle . . . 16
- 4.5 Construction Cost and Tested Scale . . . 16
- 4.6 Index Representation and Safety Contracts . . . 18

5 Representation-Aware GPU Search 20
- 5.1 Query Path and Four Distinct Ranking Objects . . . 20
- 5.2 Exact Cartesian LUT Factorisation . . . 20
- 5.3 Packed Codes and Candidate-Parallel Memory Access . . . 22
- 5.4 Exact Global Primary Top-c via Multi-Block Scanning . . . 23
- 5.5 Shared Complete-Key Thresholds and Execution Scheduling . . . 26
- 5.6 Lane-Cooperative Residual Scoring and Final Top-k . . . 29
- 5.7 IVF Probe Selection as an Independent Primitive . . . 30

6 Workload-Level Budget Calibration 32
- 6.1 Why Fixed Refinement Budgets Can Be Wasteful . . . 32
- 6.2 GT-Free Sampled Output Agreement . . . 32
- 6.3 Budget Search, Nestedness and Tie Safety . . . 34
- 6.4 Budget Baselines and Their Questions . . . 35
- 6.5 Hold-out Risk, Calibration Cost and Payback . . . 36
- 6.6 Policy Failure Modes and Applicability . . . 37

7 Experimental Evaluation **38**
7.1 Experimental Protocol . . . 38
7.2 Modern GPU ANN Recall–QPS Frontiers . . . 38
7.3 Cartesian LUT, Packed Loads and Lane Refinement . . . 41
7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation . . . 43
7.5 Held-out Budget Quality, Throughput and Payback . . . 44
7.6 Batch Regimes, G Scheduling and Workload Skew . . . 47
7.7 Cold Construction, Resident Memory and Tested Scale . . . 49
7.8 Numerical Robustness, Anomalies and Recall Attribution . . . 51

8 Discussion and Future Work **53**
8.1 Three Cross-Layer Insights . . . 53
8.2 Two Deep Negative-Result Case Studies . . . 53
8.3 Threats to Internal and External Validity . . . 54
8.4 Testable Research Directions . . . 55
8.5 Conditional Deployment Recommendation . . . 55

9 Conclusion **56**
9.1 Answers to the Four Research Questions . . . 56
9.2 Closing Principle and Outlook . . . 56

Bibliography **57**

List of Figures

1.1 Problem, mechanism and evidence . . . 2
2.1 Candidate and precision cascade . . . 4
3.1 Host and device timing boundaries . . . 10
4.1 Build and query architecture . . . 12
4.2 List-ordered physical index layout . . . 15
4.3 One- and two-pass construction . . . 17
4.4 Separate construction-allocation lifetimes . . . 18
5.1 Query stages and selection boundaries . . . 20
5.2 Packed candidate-parallel access . . . 23
5.3 Local coverage and truncated pairwise merging . . . 25
5.4 Shared complete-key filtering . . . 28
5.5 Sixteen-lane residual refinement . . . 29
6.1 Work affected by the refinement budget . . . 32
6.2 Workload-level calibration and evaluation . . . 33
6.3 Nested and non-nested candidate pools . . . 35
7.1 E1 recall–throughput frontiers . . . 39
7.2 TF32 and FP32 precision frontiers . . . 40
7.3 Small-batch competitive regions . . . 41
7.4 Cartesian lookup-table grouping . . . 42
7.5 Packed access and refinement controls . . . 43
7.6 Held-out risk across sample sizes . . . 46
7.7 Finite-pool risk prediction and observation . . . 47
7.8 Direct cumulative calibrated-reuse time . . . 48
7.9 Batch-dependent execution regimes . . . 48
7.10 Recorded construction costs . . . 50
7.11 Resident-memory decomposition . . . 50
7.12 Large-dataset search frontiers . . . 52
8.1 Mailman and rank-oriented coding . . . 54

List of Tables

1.1 Technical contributions and evidence boundaries . . . 3
2.1 Design alternatives and research gap . . . 6
3.1 Research questions and experimental controls . . . 8
3.2 Dataset and training configuration . . . 9
3.3 Principal E1 tuning dimensions . . . 10
3.4 Component implementations . . . 11
4.1 Per-vector storage model . . . 16
4.2 Analytical row-array occupancy by construction phase . . . 17
4.3 Original-input FP64 recoding audit . . . 18
4.4 Representation contracts and audit coverage . . . 19
5.1 Query-stage selection contracts . . . 21
5.2 Cartesian lookup-table trade-offs . . . 22
6.1 Budget controls and their roles . . . 36
7.1 Cross-system comparison at recall 0.97 . . . 38
7.2 FP32 sensitivity at matched configurations . . . 39
7.3 Same-session precision anchors . . . 40
7.4 Small-batch matched-recall ratios . . . 41
7.5 Refinement gains at 128 probes . . . 42
7.6 Strict computed-score selection checks . . . 43
7.7 Probe-selection throughput ratios . . . 44
7.8 Full-pool budget calibration . . . 45
7.9 Sampled-budget held-out quality . . . 45
7.10 Sampled-calibration economics . . . 46
7.11 Direct costs for 500 reuse batches . . . 47
7.12 Integrated execution and scheduling gains . . . 49
7.13 Cold construction and resident memory . . . 49
7.14 E1 allocation ranges . . . 51
8.1 Threats to validity . . . 55

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	Configured add, staged input, and independently allocated assignment rows
n_b, S_{stage}	Pipeline buffer count and byte budget per input/rotated array
Π	Square orthogonal transform; $q' = \Pi q$
B, B_r	Primary bits per subspace and residual bits per coordinate
$\rho(p), o_\ell$	Input ID at physical position p, and start of IVF list ℓ
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Sample count; allowed missing-reference slots
$C_\alpha, \mathcal{R}_\alpha$	Valid survivors; returned IDs at budget α
$\alpha_{\text{ref}}, B_{\text{reuse}}$	Reference refinement budget and number of subsequent reuse batches
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query; zero labels the one-block path
$C_{\text{buf}}, T_{\text{blk}}$	Buffer capacity; threads per block
$n_{\text{LUT}}(g_{\text{LUT}})$	LUT entries per primary subspace
$\kappa(D, p)$	Packed full key for bounds
ℓ_{ref}	Lanes cooperating on one survivor
D_{raw}	$\|q - x\|_2^2$
D_{P}	$\|q' - \widehat{p}\|_2^2$
D_{H}	$\|q' - \widehat{p} - \widehat{r}\|_2^2 + \|q'\|_2^2$

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth
FP32	IEEE binary32; GEMM mode stated separately
TF32	Reduced-precision Tensor Core inputs; FP32 accumulation
CV	$\text{sd}(n_c)/\,\text{mean}(n_c)$

1 Introduction

1.1 Context and Practical Objectives

CAGRA navigates a graph; compressed IVF scans chosen lists [18, 28, 33]. Stella shows why compact storage matters (section 7.7):

$$S_{\text{raw}} = 67.81\,\text{GiB} > S_{\text{GPU}} = \frac{32607}{1024}\,\text{GiB} > S_{\text{resident}} = 20.44\,\text{GiB}.$$

JHQ screens with primary codes and refines survivors with residuals [16]. We vary execution and survivor count on the same trained index, measuring recall, speed, memory and construction cost, including losses.

1.2 Problem Definition and Scope

For $\mathcal{X} \subset \mathbb{R}^d$, query q and selected IVF lists $\mathcal{L}(q)$, the valid-pool pipeline is

$$\begin{aligned} q' &= \Pi q, & U(q) &= \bigcup_{\ell \in \mathcal{L}(q)} \text{list}_\ell, \\ c &= \max\{k, \lceil \alpha k \rceil\}, & C_\alpha(q) &= \text{Top}^{\text{P}}_{\min(c,|U(q)|)}\, U(q), \\ & & \mathcal{R}_\alpha(q) &= \text{Top}^{\text{H}}_k\, C_\alpha(q). \end{aligned}$$

Top selection uses each computed score and its tie rule. The refined score includes a query-only constant (section 2.3); GT measures raw-neighbour recovery.

GPU construction, layout and search retain JHQ's square rotation and Cartesian primary/residual codes. One unlabelled-agreement multiplier is reused on a stable workload; it is neither per-query learning nor an absolute-recall certificate.

E1 tests squared-L2 top-10 on one RTX 5090, four datasets and batch 1,000. Six datasets support component/budget tests; T5 adds batches 8/32 (section 7.2.6). Search stays GPU-resident with host preparation/transfers. Updates, host paging, multi-GPU and billion-scale search are untested.

1.3 Three Linked Technical Challenges

Figure 1.1 connects three challenges to the mechanisms and evidence used to address them.

First, the primary codebook has useful mathematical structure that a direct GPU port can overlook. Each eight-bit subspace code indexes 256 distances. Its Cartesian coordinates permit an exact 16+16 factorisation, but more aggressive splitting adds lookups and arithmetic. The challenge is to exploit separability without assuming that the smallest table gives the fastest scan. Candidate-parallel word access and residual lane cooperation also need different data mappings.

Second, local quotas can lose global winners. For disjoint groups U_1, U_2 and $c = 4$, consider

$$\begin{aligned} \text{Top}_4(U) &\subseteq U_1, \\ \left|\text{Top}_4(U) \cap \left(\text{Top}_2(U_1) \cup \text{Top}_2(U_2)\right)\right| &= 2 < 4. \end{aligned}$$

Groups need coverage before merging. Bounds reduce maintenance; scheduling controls parallelism. IVF routing is separate.

Third, reference agreement cannot fix reference error. A $k = 1$ counterexample is

$$\mathcal{R}_\alpha = \mathcal{R}_{\text{ref}} = \{a\}, \quad \mathcal{T}_1 = \{b\}, \quad a \neq b \implies D_S(\alpha) = 0, \quad \text{Recall@1} = 0 \qquad (S = 1).$$

We test quality alongside calibration/reuse cost.

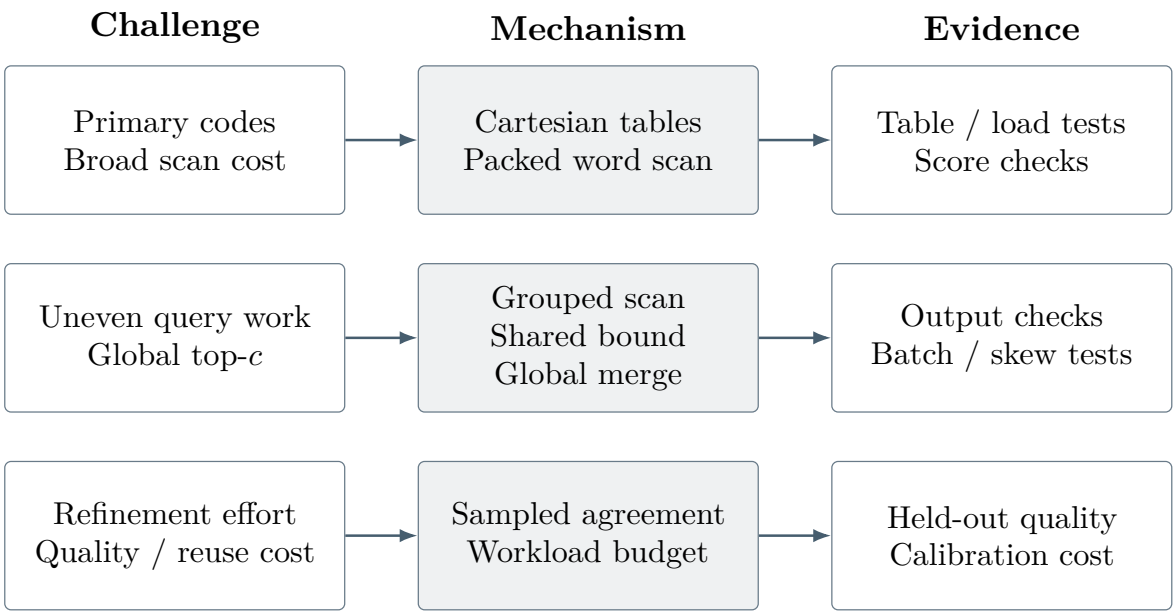


Figure 1.1. Problem, mechanism and evidence. The arrows organise research questions; they do not imply that independently measured speedups multiply or that an unlabelled agreement rule guarantees GT recall.

1.4 Research Questions and Falsifiable Claims

RQ1: Construction and capacity. How can JHQ be built on a GPU, and what cost, resident capacity and build peak result (section 7.7)?

RQ2: Correct and efficient execution. Which execution choices speed up search while preserving score/tie contracts (sections 7.3, 7.4 and 7.6)?

RQ3: Budget quality and total cost. Can unlabelled calibration preserve held-out quality and repay its cost (section 7.5)? The GT-labelled oracle is an evaluation control.

RQ4: Competitive operating regions. Under host-to-host timing, where does JHQ lead at matched recall? The answer uses independently tuned E1 frontiers and explicit memory definitions (section 7.2). Int8 graph competitors, high-recall reversals and absent coverage are part of the answer.

Chapter 3 turns these questions into controlled comparisons. Together they separate what was built, whether its execution is correct, what effort a workload requires, and whether the resulting system has useful operating regions.

1.5 Research Contributions

Table 1.1 states three linked project contributions and their inherited foundations. The core contribution is the adaptation and evaluation of a particular representation–execution–budget combination. JHQ, Cartesian quantisation, IVF, bitonic/radix selection, merging and CUDA Graph execution retain their prior-art status.

Table 1.1. Technical contributions and their evidence boundaries.

	Inherited foundation	Project contribution	Evidence
C1	JHQ rotation, codebooks and hierarchy	GPU training/encoding, bounded batch construction and list-ordered packed representation	Search of 17.78M vectors at 20.44 GiB resident; repeated train+add 10.54–10.65 s; sampled construction peak 22.58 GiB
C2	Cartesian additivity and established GPU primitives	Integrated factorised scoring, complete-key grouped retention/bounds, lane refinement and execution scheduling	Two-group LUTs lead in 22/24 cells; scheduled execution 2.727× at batch 8; explicit correctness scope
C3	Candidate expansion and output-agreement ideas	Workload-level budget rule, conditional selection analysis, held-out controls and cost/risk evaluation	FULL meets the relative-loss target in 12/12 cells; smaller samples expose risk; four cells directly measure calibrated reuse

2 Background and Related Work

2.1 ANN Objective and Metrics

Exact neighbours in stable-ID database $\mathcal{X} \subset \mathbb{R}^d$ minimise

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

Recall@10 counts stored-GT overlap, including GT ties (equation (3.1)).

Chapter 3 defines QPS, batch latency, build time and memory boundaries. Batch calls do not measure concurrent-request tails.

2.2 IVF, PQ, ADC and Refinement

IVF routes to n_{probe} lists:

$$U(q) = \bigcup_{\ell \in \mathcal{L}(q)} \text{list}_\ell, \qquad n_c = |U(q)|.$$

A neighbour outside U cannot be recovered by later scoring.

PQ learns codebooks by subspace [17]. With unquantised q, ADC uses

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

Lookups replace coordinate scoring. A learned subcodebook need not itself be Cartesian; Chapter 5 relies on that extra structure.

The residual definitions differ:

$$r_{\text{IVF-PQ}} = x - \mu_{\ell(x)}, \qquad r_{\text{JHQ}} = \Pi x - \widehat{p}(x).$$

JHQ therefore adds no IVF centroid during reconstruction. Candidate expansion/refinement is established [10, 18].

With $|U(q)| \geq k$ and $c = \max\{k, \lceil \alpha k \rceil\}$,

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

For GT set $\mathcal{T}_k$, set inclusion gives

$$|\mathcal{R}_\alpha \cap \mathcal{T}_k| \leq |C_\alpha \cap \mathcal{T}_k| \leq |U \cap \mathcal{T}_k|.$$

Loss attribution requires fixed-universe counterfactual scores.

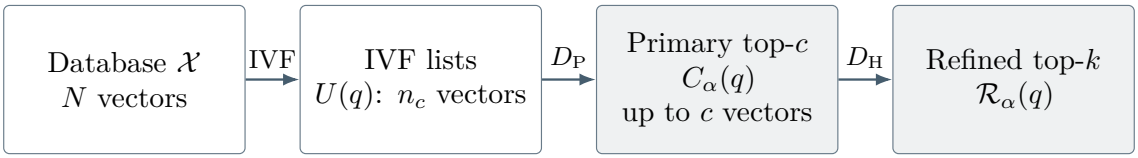


Figure 2.1. Candidate and precision cascade. IVF selects lists; primary scoring selects vectors; only survivors read residuals. The hierarchy is inherited. This diagram defines logical sets and score fidelity, not measured recall losses or GPU kernel boundaries.

$$W_{\text{primary}} = O(n_c M) \text{ lookups}, \qquad W_{\text{refine}} = O(cd) \text{ coordinates}.$$

Probes change U; α changes retention. Access/selection add latency.

2.3 JHQ Representation and Assumptions

2.3.1 What is inherited

JHQ uses square $\Pi \in \mathbb{R}^{d\times d}$ [16]:

$$\begin{aligned}\Pi^\top\Pi = I \implies \|\Pi(q-x)\|^2 \\ = (q-x)^\top\Pi^\top\Pi(q-x) = \|q-x\|^2.\end{aligned} \tag{2.4}$$

Dimension stays d. Finite-precision geometry, GEMMs and quantisation are checked separately.

Partition y into M subspaces of dimension $D_s = d/M$. With B bits per primary subspace, $K = 2^B$ labels are formed from $L = 2^{B/D_s}$ scalar levels, requiring the implemented Cartesian parameters to make L an integer. Writing Φ for the standard normal CDF, the constructor uses

$$a_i = \sigma\Phi^{-1}\left(\frac{i-1/2}{L}\right), \quad i = 1,\dots,L, \qquad \mathcal{C}_m = \{a_1,\dots,a_L\}^{D_s}. \tag{2.5}$$

At the evaluated $B = D_s = 8$, $L = 2$ and $\mathcal{C}_m = \{-a,+a\}^8$, with $a \simeq 0.67449\sigma$. Each byte records eight coordinate signs. The implementation estimates σ from vector energy; unit-normalised inputs motivate the scale $1/\sqrt{d}$. This representation is prior art, not an invention of the GPU execution layer.

Residuals are $r = y - \widehat{p}$, giving reconstruction $\widehat{y} = \widehat{p} + \widehat{r}$. The evaluated codes use $d/8$ primary bytes and d residual bytes per vector, before metadata. Refining fewer rows reduces reads, not stored residual size.

2.3.2 Model assumptions and score semantics

Random rotation can spread coordinate energy, motivating a shared Gaussian-inspired scalar model. It does not force an arbitrary realised database to have independent, zero-mean Gaussian coordinates. For a fixed rotation and empirical database mean μ, the transformed mean is $\Pi\mu$ and $\|\Pi\mu\| = \|\mu\|$. Averaging over random rotations is different from centring one fixed corpus. Even a uniformly random point on a sphere has dependent coordinates through its fixed norm. Consequently, isometry is an algebraic guarantee; the distributional approximation is a modelling assumption. Chapter 8's centred-coding pilot examines one practical consequence of that distinction.

The inherited primary levels are Gaussian mid-quantiles. For $X \sim \mathcal{N}(0,\sigma^2)$ and a zero threshold,

$$\begin{aligned}a_{\mathrm{MSE}} &= \arg\min_a \mathbb{E}[(X-a)^2 \mid X > 0] \\ &= \mathbb{E}[X \mid X > 0] = \sigma\sqrt{2/\pi} \simeq 0.79788\sigma.\end{aligned}$$

For fixed signs and positive scale, however,

$$D_{\mathrm{P}} = \|q'\|^2 + da^2 - 2a\langle q', s\rangle, \qquad s \in \{-1,+1\}^d. \tag{2.6}$$

For fixed signs,

$$a > 0 \implies \arg\min_x D_{\mathrm{P}}(q,x) = \arg\max_x \langle q', s(x)\rangle.$$

Residuals still depend on scale; separability alone gives no optimality.

The scores use different vectors:

$$\begin{aligned}D_{\mathrm{raw}} = \|q-x\|^2, \qquad D_{\mathrm{P}} = \|q'-\widehat{p}\|^2. \\ D_{\mathrm{H}} = \|q'-\widehat{p}-\widehat{r}\|^2 + \|q'\|^2.\end{aligned} \tag{2.7}$$

Thus, for a fixed query,

$$\arg\min_x D_{\mathrm{H}} = \arg\min_x \|q'-\widehat{p}-\widehat{r}\|^2,$$

while raw-vector recall remains empirical.

2.4 GPU Execution Concepts

Warps have 32 threads with bounded block resources [25]. JHQ scans candidates and refines coordinates in parallel; extra blocks duplicate state.

Faiss uses warp selection [18]; bitonic/radix/merge choices trade state for work, alongside global selectors AIR/GridSelect [38].

2.5 Graph Search on GPUs

SONG parallelises graph search [39]; CAGRA adjusts connectivity/search width [28]. We compare fp32 and quantised CAGRA.

BANG uses host graphs to extend capacity [37]; JHQ retains compressed residency.

2.6 Quantised Search and Fast Tables

2.6.1 Learning a representation versus exploiting its structure

OPQ may lose Cartesian separability [13]; ScaNN targets inner-product error [15]. Reconstruction error and neighbour order can disagree.

2.6.2 Small lookup tables have substantial prior art

Fast Scan/Quicker ADC use CPU register tables [1, 2]. JHQ splits GPU Cartesian tables and checks FP32 ties.

RaBitQ's estimator [11] and multi-bit extension [12] have their own guarantees. GPU IVF-RaBitQ provides the fused-search baseline [33].

2.6.3 Refinement effort and adaptive stopping

Faiss/cuVS expose refinement effort [7, 10]. Other policies use GT [9], learned stopping [19] or recall targets [6]; ours compares workload outputs.

2.7 Research Gap and Design Choices

Table 2.1 compares the alternatives by the constraint they address. The literature already provides GPU graph search, compressed distance computation, register-oriented lookup, fast selection and adaptive effort. A useful contribution must therefore specify the representation contract, the integration decision and the evidence for it. "A GPU version" alone does not identify that contribution.

Table 2.1. Design alternatives and the thesis's narrower research gap. This is a mechanism comparison; measured rankings follow in Chapter 7.

Alternative	Useful structure	Principal trade-off	JHQ-GPU question
IVF-PQ / OPQ	Learned subspace codes; ADC	Distortion and training; arbitrary subcodebooks couple coordinates	What does an internally Cartesian primary code permit in GPU scoring?
Fast Scan / Quicker ADC	Small tables and SIMD lookup	Table precision, shape and ISA constraints	Does exact splitting repay extra GPU lookups?
SONG / CAGRA	Selective graph navigation	Dependent access, graph storage and construction	Where does a resident hierarchical scan compete?
BANG	Compressed GPU scoring with host graph	Capacity versus communication	What can complete compressed residency achieve?
IVF-RaBitQ	GPU code construction and fused search	Bit budget, estimator and probe effort	Which recall regions justify the residual hierarchy?
Adaptive effort	Refine ratios, autotuning, learned or declarative stopping	Labels, reference fidelity, prediction or calibration cost	Can unlabelled workload agreement select useful effort at an acceptable risk and total cost?

At fixed representation, we test execution correctness, scan parallelism and refinement quality/cost.

3 Research Design and Measurement

Paired execution, held-out calibration and system frontiers use different controls, defined here.

3.1 Research Questions and Comparisons

Table 3.1 defines the comparison unit and what would challenge each research claim. A *cell* fixes a dataset, index configuration and workload setting; arms are alternative implementations or budgets within that cell. For paired execution measurements, the index state, queries, probe count, refinement budget and scoring representation are held fixed unless the named treatment changes them. The recorded paired ratio is $Q_{\text{treatment}}/Q_{\text{control}}$. Across J stated cells, equal weighting gives

$$G = \exp\left(\frac{1}{J}\sum_{j=1}^{J}\log\frac{Q_{\text{treatment},j}}{Q_{\text{control},j}}\right).$$

Table 3.1. Research questions as testable comparisons. A capacity observation has a different control from a paired execution intervention.

Question	Intervention / comparison	Held fixed or accounted for	Measure and counter-evidence
RQ1: build and capacity	GPU construction; cached versus cold lifecycle; compressed versus raw storage	N, d, representation, routing and timed construction boundary	Train/add time and resident bytes; failed builds or model–allocation gaps delimit feasibility
RQ2: execution	LUT grouping, word access, refinement lanes, grouped scan, shared bound and probe selection	Paired state, workload and score; reference output semantics	Wall QPS, stage time and output checks; slowdowns or non-tie mismatches challenge the mechanism
RQ3: budget	Sampled / full-pool rule versus fixed 64, 100, 200, enumeration and GT oracle	One index and route; disjoint calibration and held-out query positions	Held-out recall loss, selected-arm QPS and calibration cost; threshold crossings challenge reliability
RQ4: operating region	Independently tuned JHQ, CAGRA, IVF-PQ and IVF-RaBitQ	Exported vectors, GT, k, device and host-to-host boundary; memory reported separately	Matched-recall QPS and coverage; stronger competitors at other recall targets challenge broad superiority

Ablations estimate speed effects, not unmeasured cache/occupancy changes. System comparisons are separate (section 7.6).

3.2 Datasets, Queries, Ground Truth and Splits

Table 3.2 records the six exported datasets used by the newer budget, LUT and scale campaigns. The poster's E1 comparison uses the first four. Vogue contains fashion-image embeddings; arXiv contains abstract embeddings; the two OpenAI datasets use Qdrant's DBpedia exports [5, 31, 32, 34]. BGE uses the Italian Wikipedia configuration, and Stella uses the TREC24 BioGen corpus export [35, 36]. Sizes describe the measured exports after preparation, rather than the current upstream cards.

Table 3.2. Data and JHQ training configuration. n_q is the actual query count; $n_t = 39n_{\text{list}}$ is the requested coarse-training sample size. E1 uses the first four rows. All rows use squared L2 and $k = 10$.

Dataset	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

OpenAI/BGE/Stella preparation removes non-finite rows and uses

$$x \leftarrow x/\|x\| \quad (\|x\| > 10^{-12}).$$

Seed 42 samples without replacement. Stella uses 999 corpus queries, replacing the 65-query benchmark; relevance labels are absent. OpenAI checks support alignment for all queries/10,299 base rows without original-ID joins (sections 7.2 and 7.2.3).

Methods share exported query/GT files. Exhaustive tiled squared-L2 search over FP32 vectors maintains GT top-k, including its ties:

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

Here $\mathcal{T}_k$ is the stored GT prefix and $\mathcal{R}_k$ the returned ID set. GT widths are 100 (Vogue/arXiv) or 20 (others); only ten entries count. Duplicate IDs earn no extra credit. Range/duplicate/repeat checks are separate.

JHQ uses $M = d/8$, $B = B_r = 8$, eight coarse iterations and 100,000 residual-training rows. Sweeps reload/re-encode; cold builds refit. Training uses database rows.

$$\mathcal{Q}_{\text{cal}} = \{q_i : i \bmod 2 = 0\}, \qquad \mathcal{Q}_{\text{hold}} = \{q_i : i \bmod 2 = 1\},$$

$$(|\mathcal{Q}_{\text{cal}}|, |\mathcal{Q}_{\text{hold}}|) = \begin{cases} (500, 499), & \text{Stella}, \\ (500, 500), & \text{otherwise}. \end{cases}$$

Seeded subsamples of sizes 32, 64 and 128 are drawn 64 times per cell; FULL uses all 500 calibration queries. Labels assess the oracle and holdout; agreement uses outputs (sections 3.5.3 and 6.2).

3.3 Timing, Variance and Memory Accounting

The measured interval is the host search call:

$$t_{\text{call}} = t_{\text{return}} - t_{\text{entry}}, \quad \begin{cases} \text{entry: FP32 queries in host memory,} \\ \text{return: host IDs/scores after stream completion.} \end{cases}$$

Staging, transfers, query work and int8 query quantisation are timed; build/fitting are excluded (figure 3.1). Handles/outputs are reused. JHQ uses pageable/C++, CAGRA/PQ pinned/Python, RaBitQ pinned/C++; these costs matter to small margins.

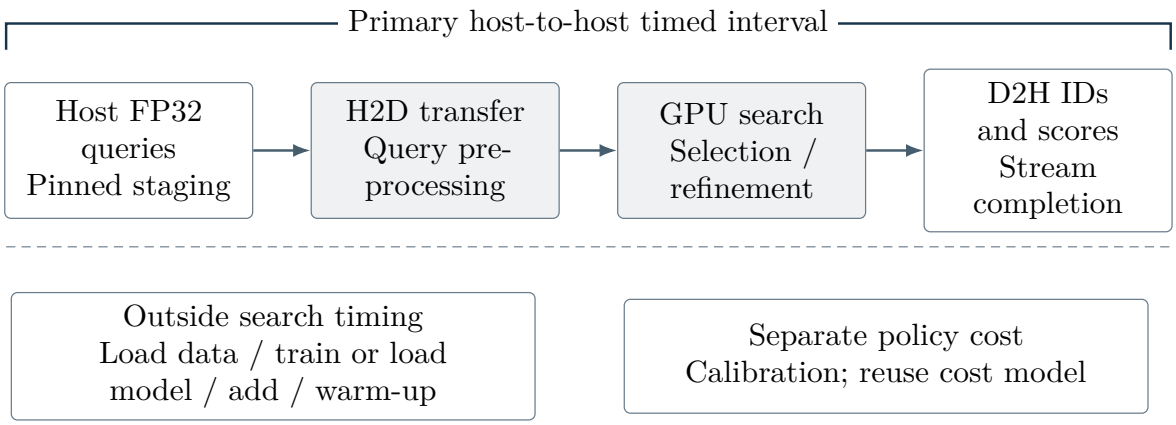


Figure 3.1. Timing boundaries. The host interval includes transfers and query-specific computation. Device-stage profiles explain parts of that interval; separate profiling and synchronisation can change execution, so their sum is not an interchangeable end-to-end latency.

Times are in seconds:

$$\bar{t}_r = \frac{1}{m_r}\sum_{i=1}^{m_r} t_{ri}, \qquad Q_r = \frac{b}{\bar{t}_r}, \qquad Q = \operatorname{median}_r Q_r. \tag{3.2}$$

E1: batch 1,000, five rounds, normally 24 calls. JHQ warms once per variant/round; baselines three times initially. Slow baseline points use fewer calls. JIT/capture are excluded; call percentiles do not describe concurrent requests.

Slow valid calls stay. For illustrative call times $(1, 1, 10)$ ms,

$$\bar{t} = 4\,\text{ms} \neq \operatorname{median}(t) = 1\,\text{ms}, \qquad b/\bar{t} \neq b/\operatorname{median}(t).$$

Shared indexes/pools do not provide independent builds.

RTX 5090/32,607 MiB; driver 595.71.05; CUDA 13.0; cuVS 26.08.01. OpenMP uses 16 of 25 CPUs; E1 records 575 W. Clocks/load can vary. JHQ JITs compute_80 PTX [27]; library builds/device profiles are separate.

Resident totals include model, workspace and context. CAGRA index deltas and JHQ context-subtracted totals differ (sections 4.4 and 7.7). Pools/scratch prevent peak matching. Build peaks use 20 ms sampling; train/add excludes loading/host setup.

Supplementary sessions. Campaigns are separate (section 3.5.3); cached-state/precision changes follow sections 4.2 and 7.2.3. Historical baseline ratios remain cross-session.

3.4 Baselines and Frontier Construction

CAGRA fp32/int8, IVF-PQ and IVF-RaBitQ [17, 28, 33] use separate grids (table 3.3); FP32 brute force is exhaustive. Storage/reconstruction fidelity may differ. Consequently the main frontier measures achievable speed at a quality target within these grids. A separate memory account accompanies it.

Table 3.3. Principal E1 tuning dimensions. Exact per-point parameters and coverage extensions remain in the raw records. The grid is finite and differs between methods.

Family	Recorded search dimensions
JHQ	Probes and α; initial probes 64/128/256 and 0.5/1/2 times the workload budget; coverage extensions reach probe 1,024 and larger/smaller budgets
CAGRA fp32/int8	Graph degree 32/64; search width 1/2/4/8; intermediate top-k capacity 32–512
IVF-PQ	Eight-bit subcodes; 192/384/768-byte codes at $d = 768$, 384/768/1,536 at higher dimensions; fp16/fp32 lookup settings; probes 16–512
IVF-RaBitQ	4/8/9 bits per dimension, supported search modes, probes 16–1,024; additional 4,096-list OpenAI indexes
GPU brute force	Exhaustive FP32 squared-L2 search, common host boundary

Dominance and the measured frontier are

$$a \succ b \iff R_a \geq R_b,\ Q_a \geq Q_b,\ (R_a, Q_a) \neq (R_b, Q_b).$$

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

No qualifying measured point means undefined. For adjacent frontier points,

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

Within 5×10^{-4}, the parser snaps; otherwise it interpolates without extrapolation. Qualifying endpoints remain eligible in T5. Near ties report both estimators; memory is unmatched.

Failed/invalid outputs are excluded; repeat-ID flags alone are not. Isolated reruns replace contaminated builds; skipped Stella batch-1,000 runs add no points (section 3.5.2). Int8 remains included (figure 7.1); ADC/poster plots have separate scopes.

3.5 Experimental Records and Reproducibility

Records retain versions/settings/times/outputs. Paired runs share states; state/precision changes define separate families. Missing historical binary/model hashes limit exact reproduction (section 3.5.1). Score/tie correctness differs from raw recall, which also depends on routing/reconstruction. Sampling/ordering/measurement limits are in sections 6.2, 6.3 and 8.3.

3.5.1 Sources and available records

The project archive holds source/measurement snapshots and selected T1, T4/T5 and A2 inputs. Complete vectors, trained states and historical executables are not all available. T4 candidate dumps were deleted; retained summaries cannot support an independent resort.

3.5.2 Additional timing conventions

Warm-up reduces E1 call counts at 274 points; initial conversion/validation differs from steady-state timing. The 369 repeat-ID flags include tied reordering; 401 also counts cross-protocol differences. Flags alone exclude no points, and JHQ lacks the same flag.

3.5.3 Calibration and follow-up contracts

The campaign sets the 200-start grid explicitly. Fixed arms use one warm-up/three timed calls. Calibration times full searches and $O(Sk^2)$ host comparison, excluding gathering/scan calibration and bypassing graph capture. Warm-up visits every query without passing outputs/GT to agreement. Callers install the returned index with `set_alpha(A[index])`; selected-arm rates reuse fixed-arm timing (section 6.5). For 64 sorted losses, the recorded quantile is

$$L_{95} = L^{\text{sorted}}_{\lfloor 0.95(64-1) \rfloor} = L^{\text{sorted}}_{59} \qquad \text{(zero-based)}.$$

Follow-up sessions have separate states/boundaries unless stated. A2 corroborates the PTX contract (section 5.5); T4's score-export scope is in section 7.4.1. T5 includes null calls/int8 query quantisation, with API/warm-up differences (section 7.2.6).

Table 3.4. Implementation families used by component tests. Each experiment retains its own control; versions are not successive whole-system speedups.

Version	Mechanism / experimental role
v61	Word-load and lane-refinement controls
v62 / v63	Grouped full-key selection / shared bound
v65 / v69	Batch/skew scheduling / radix probe selection
v69 / v71 / v73	Budget calibration / LUT sweep / strict-score export
v69 / v72	OpenAI: TF32 versus FP32 add/query arithmetic on the same trained cache

4 GPU Index Design and Construction

Training/encoding produces bounded-memory, list-ordered GPU arrays [16]; chapter 5 and section 7.7 give search and cost.

4.1 End-to-End Build and Search Boundaries

Train fits models; add encodes/list-orders rows; search uses the completed index. Cache hits skip fitting, not encoding, so cold-build and cached-search costs differ.

Hosts prepare samples/rotation/primary codebooks; GPUs rotate, fit IVF/residual models and encode bounded batches into list order.

Models hold rotation, codebooks and centroids; database state holds offsets, IDs, codes and corrections. Graphs reference reused query buffers.

One successful `add` fixes positions. Further adds are rejected; updates require rebuilding. Mutable/transactional publication is unimplemented.

Host IDs/scores and call timing follow equation (5.20) and section 3.3.

4.2 Training, Rotation and Codebook Generation

4.2.1 Transform and sampling

Host QR of a seeded Gaussian matrix produces square Π:

$$Y_t = X_t \Pi^\mathsf{T}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^\mathsf{T}\Pi = I. \tag{4.1}$$

C0 checks finite-precision QR/GEMM error; dimension remains d (section 4.6).

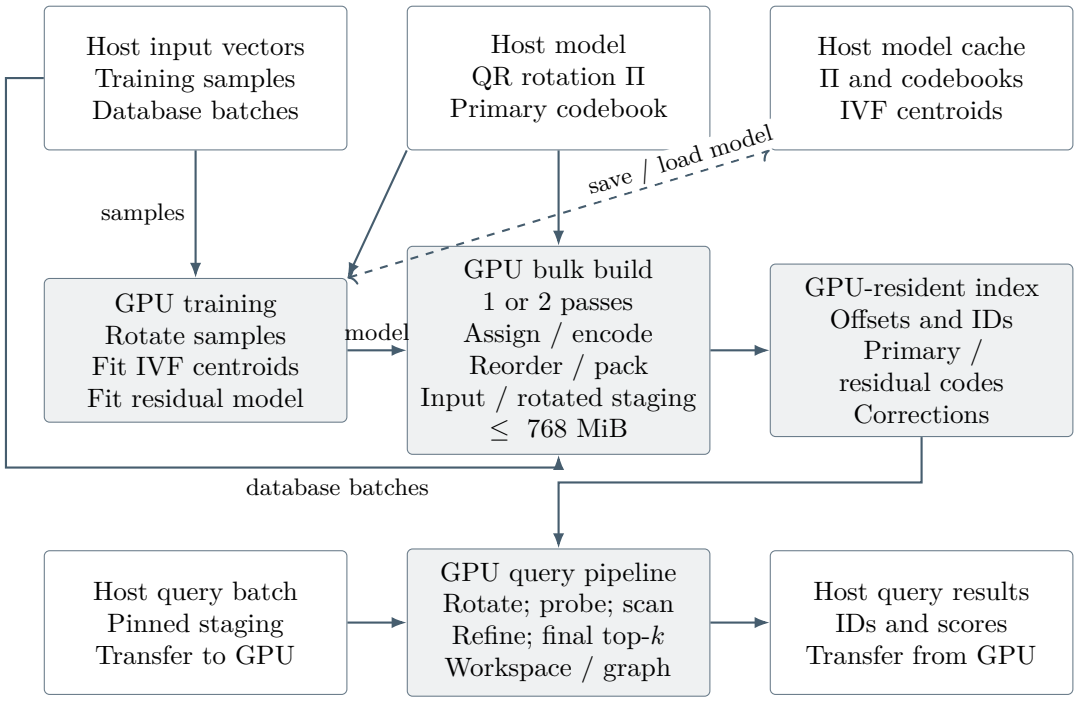


Figure 4.1. Implemented build and query boundaries. Host work includes sampling, orthogonal-matrix construction and analytical primary-codebook generation; the evaluated configuration fits IVF and residual models on the GPU. The cache stores trained state, whereas the completed database representation resides on the device. Dashed arrows: model reuse. The staging bound excludes other scratch.

Strided primary/IVF training takes $n_t = 39n_{\text{list}}$:

$$n_t = \begin{cases} 159{,}744, & \text{Vogue,} \\ 319{,}488, & \text{arXiv/OpenAI,} \\ 1{,}277{,}952, & \text{BGE/Stella.} \end{cases}$$

This is Faiss's warning threshold, not a quality guarantee [8]. Residual fitting uses 100,000 rows.

An older twelve-cell Vogue/OpenAI-3072 sweep ($B_r = 4/8$, probes 32/128/256) fixed primary/IVF models. With 25 Lloyd iterations, 100K rather than all rows lost at most 0.0018 recall (eleven losses/one tie). Intermediate sizes were non-monotone; this supports bounded training only within that protocol.

4.2.2 Why retain the analytical primary model?

Sample scale and Cartesian levels are

$$\sigma^2 = \frac{1}{n_t d} \sum_{i=0}^{n_t-1} \|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

$$B = D_s = 8 \Longrightarrow \mathcal{C}_m = \{-a, +a\}^8, \quad a = 0.674\sigma.$$

Eight signs fit a byte, enabling $O(d)$ encoding/split LUTs [16] (section 5.2). Arbitrary Lloyd updates may break Cartesian structure.

For fixed sign vector $s \in \{-1, +1\}^d$,

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

$$a > 0 \Longrightarrow \arg\min_s \|q' - as\|^2 = \arg\max_s \langle q', s\rangle.$$

The order needs $a > 0$, not Gaussian data. Residuals depend on a; zero scale/FP32 ties are separate cases.

4.2.3 Coarse assignment as filter and refine

IVF initialises spaced rotated rows and runs eight iterations using $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$. Supported dimensions use int8 filtering, resolving ambiguity in FP32.

Lemma 4.1 (Assignment with certified score intervals)**.**
Let $\widetilde{y}, \widetilde{\mu}_\ell$ be dequantised approximations, $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$. Keeping centroid norms exact, the approximate score $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell\rangle$ has real-arithmetic error bounded by

$$E_\ell = 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|). \tag{4.4}$$

Given valid intervals $[L_\ell, U_\ell]$ containing every target score, discarding only $L_\ell > \min_j U_j$ and exactly resolving the remaining scores preserves the minimum and its specified tie rule.

Proof.
Using $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$,

$$\langle y, \mu_\ell\rangle - \langle \widetilde{y}, \widetilde{\mu}_\ell\rangle = \langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle,$$
$$|s_\ell - \widetilde{s}_\ell| \le 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|) = E_\ell.$$

For $j^* \in \arg\min_j U_j$,

$$L_\ell > U_{j^*} \Longrightarrow s_\ell \ge L_\ell > U_{j^*} \ge s_{j^*}.$$

The strict inequality excludes ℓ; equality stays eligible. □

FP32 headroom lacks a bitwise certificate without accumulation-error bounds. FP16 differs on 0/14,838 Vogue, 6/31,748 BGE and 3/16,359 Stella tail assignments. Cold logs do not identify precision.

4.2.4 Residual fitting and the iteration budget

Residuals stay $r = \Pi x - \widehat{p}(x)$. Per subspace, GPU sorting/Lloyd fits $K_r = 2^{B_r}$ sorted scalar levels shared across coordinates, capped at 2,000 updates.

Historical 100K training gained 0.0063–0.0076 recall at 25→2,000 updates. Tail-bin/BGE/Stella changes persisted: the cap is not convergence, without contradicting exact Lloyd descent. At 20,000 updates, historical Stella all-row/100K recall was 0.9921/0.9920.

4.2.5 Construction work and observed imbalance

$$W_{\mathrm{QR}} = O(d^3), \qquad W_{\mathrm{rotate}} = O(n_t d^2).$$

Direct coarse assignment costs $O(n_t n_{\mathrm{list}} d)$ per iteration.

$$W_{\mathrm{primary}} = O(MKD_s), \qquad W_{\mathrm{residual\ extraction}} = O(n_r d),$$

followed by sorting and scalar updates. These describe work, rather than predict latency. Synchronised phase diagnostics place about 83% of BGE/Stella training in IVF k-means (about 2.97 s). They use one add buffer and forced synchronisation, separately from the normal overlapped timings in section 7.7.

List-size imbalance enters the scan rule in section 5.5. The individual contributions of eight IVF iterations, spaced seeds and finite sampling to that imbalance were not isolated.

Arithmetic used in the measured implementation. The evaluated implementation (v69) stores FP32 matrices but enables cuBLAS TF32 tensor-core math for $d \geq 1{,}536$. In its compiled faithful branch, a null override falls through to the dimension-based default. This affects training/add/query rotation and centroid GEMMs on both OpenAI datasets; an environment override cannot disable that branch. Thus FP32 storage does not imply full FP32 multiplication. The supplementary FP32 variant (v72) corrects this switch. Its sensitivity experiment reuses the same TF32-trained cache and changes add/query arithmetic, rather than retraining the model in FP32 (section 7.2.3).

4.3 IVF Assignment, IDs and Physical Memory Layout

4.3.1 From database IDs to list-ordered positions

An input row has identifier $i \in \{0, \ldots, N-1\}$. Coarse assignment places its transformed vector in a list according to the computed centroid scores. The mathematical assignment target is

$$a(i) = \operatorname*{arg\,min}_{0 \leq \ell < n_{\mathrm{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

Its implementation uses the selected numerical assignment path. Encoding the primary and residual representations does not subtract $\mu_{a(i)}$; list membership restricts the future search universe independently of those reconstructions.

The builder sorts assignments together with a sequence of input IDs. Let $\rho(p)$ be the input ID at sorted physical position p and let o_ℓ be the start of list ℓ. The completed arrays satisfy

$$0 = o_0 \leq o_1 \leq \cdots \leq o_{n_{\mathrm{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

Lower-bound searches give offsets, with

$$o_\ell = o_{\ell+1} \Longleftrightarrow \text{list } \ell \text{ is empty.}$$

Positions are stable within a snapshot, not across builds. All arrays use the same $\rho(p)$.

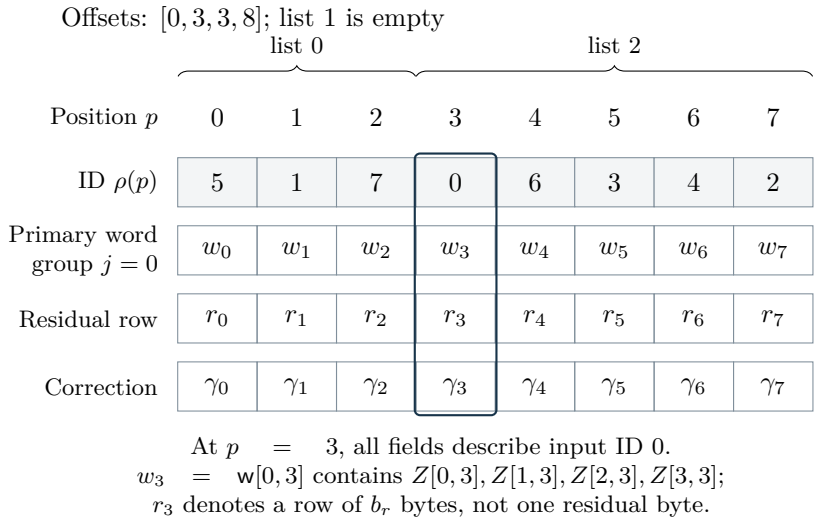


Figure 4.2. One common physical position joins IDs, packed primary words, residual rows and correction factors. This illustrative eight-row index has offsets $[0, 3, 3, 8]$, including an empty list. Words pack subspaces of one vector; values are schematic.

IDs map $p \mapsto \rho(p)$. Two-pass add temporarily builds ρ^{-1} for scatter. Primary ties use positions; final ties use database IDs.

4.3.2 Exact addresses and representation widths

For primary byte $Z[m, p]$,

$$\mathrm{offset}_{\mathrm{byte}}(Z[m, p]) = mN + p.$$

Final packing produces

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \mathrm{addr}_{\mathrm{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

Here $j = 0, \ldots, M/4 - 1$. word-layout build $\Longrightarrow$ $M \bmod 4 = 0$; other M are rejected. A separate byte-layout compilation retains the byte representation. Each packed array occupies NM bytes, but repacking temporarily holds both old and new copies, costing $2NM$. The one-pass builder temporarily gathers a vector-major $[N, M]$ primary array, transposes it, and then repacks; each obsolete copy is freed. The two-pass builder scatters directly into the subspace-major array before the same final repack. The search-time lane mapping is discussed in Section 5.3.

Residual codes remain row-major, with $b_r = \lceil dB_r/8 \rceil$ bytes per vector. In the eight-bit configuration, coordinate t of row p is at byte offset $pb_r + t$. In the four-bit path it is in byte $pb_r + \lfloor t/2 \rfloor$, with even coordinates in the low nibble and odd coordinates in the high nibble. A four-byte correction per physical row stores

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

Its value must move with that row's codes during gathering or scattering.

List-order primary scans fetch residuals only for survivors. Raw FP32 reranking would add $4Nd$ device bytes. Here compressed reconstructions are refined; Chapter 7 measures recall.

4.4 Resident Storage, Workspace and Index Lifecycle

4.4.1 Stored bytes and allocated bytes

For byte-sized primary codes,

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

IDs/corrections contribute $8N$ bytes. With $M = d/8$, $B_r = 8$,

$$b_{\text{code}} = d/8 + d = 9d/8 = 1.125d.$$

The code payload exceeds SQ8 by 12.5%; graph-free need not mean smaller than int8.

At graph degree $d_g = 32$, CAGRA uses $d + 4d_g$ row bytes. Stella JHQ payload (19.20 GiB) is close to int8 CAGRA's 19,656 MiB, under different scopes. α changes residual reads, not stored bytes.

Basic model storage is

$$S_{\text{model}} = 4d^2 + 4MKD_s + 4MK_r + 4n_{\text{list}}(d+1) \quad \text{bytes.}$$

Derived arrays are additional. Workspace holds two query matrices plus centroid scores, LUTs, probes and survivors. Optional residual LUTs add $4b_{\text{cap}}dK_r$ bytes; defaults score directly. Group capacity stays about 93.8 MiB at 128 probes even when unused (equation (5.13)).

Input is caller-owned. Separate staging/order storage is

$$S_{\text{pinned}} = 4n_b a_{\text{in}} d, \qquad S_{\text{order}} = S_{\text{inverse}} = 4N.$$

Assignment scratch ends before gathering (equations (4.10) and (4.13)).

4.4.2 Reuse, recovery and invalidation

Cache keys omit code version/full-data hashes. Code/data changes need invalidation. Sweeps share states; cold builds refit. Similar recall cannot establish model identity.

Cache hits still pay QR, warm-up, model upload and add. Stella's 6.67 s add includes initial workspace, excludes scan calibration/search warm-up and is not restart time.

α changes need no retraining. Workspace grows before capture; changing batch size, survivors, probes, lanes, groups or bounds triggers recapture. Calibration may bypass capture. Index-owned buffers are reused until release.

4.5 Construction Cost and Tested Scale

4.5.1 Bounded input batches and two-pass construction

Whole-database raw/rotated storage is $4Nd + 4Nd = 8Nd$ bytes. Bounded staging avoids it. For n_b pipeline buffers, configured add rows a_{add} and per-array byte budget S_{stage}, the source stages

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

Table 4.1. Per-vector bytes. JHQ includes codes, ID and correction; SQ8 is vector codes alone; CAGRA int8 models vector codes plus 32 four-byte neighbours. Offsets, model parameters, workspace and other runtime allocations are excluded. These components are not complete process totals.

d	Primary	Residual	ID + corr.	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

$$4d \le S_{\text{stage}}, \quad n_b = 3,\ S_{\text{stage}} = 128\,\text{MiB} \Longrightarrow S_{\text{float}} \le 2n_b S_{\text{stage}} = 768\,\text{MiB}.$$

Assignment/encoder scratch is additional.

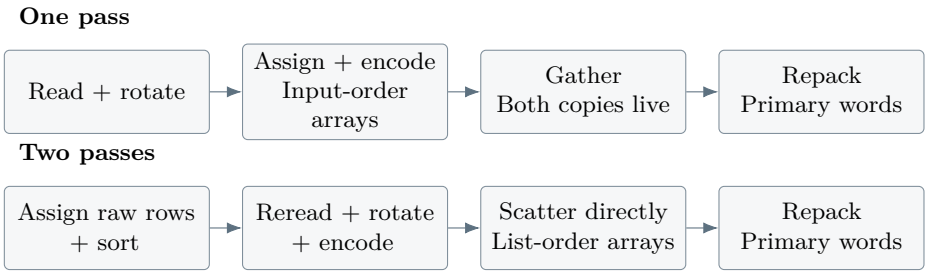


Figure 4.3. Construction paths. One pass briefly holds input-order and gathered encodings; two passes reread/rotate to encode directly into list order. Raw assignment against back-rotated centroids uses orthogonality.

4.5.2 Phase occupancy and the process peak

The one-pass estimate is

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes.} \tag{4.11}$$

Its ten live arrays sum to

$$3NM + 2Nb_r + 20N = N(3M + 2b_r + 20).$$

The 1 GiB allowance is heuristic, not a process-peak bound.

Final two-pass repack counts IDs, residuals, corrections and two primary copies:

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

Whole-build peak also considers earlier pipeline/inverse scratch, model, sort, context and later workspace allocations.

Table 4.2 quantifies the compressed-copy saving. Stella's cold log selects two passes and prints 41.6 GiB for equation (4.11), against 30.6 GiB available at the tail. The separately sampled resident process totals, including context and query workspace, are 11.93 GiB for BGE and 20.44 GiB for Stella (section 4.4). The supplementary 20 ms device samples observe a 26.08 GiB BGE peak near the start of add, and a 22.58 GiB Stella peak during repack. These are sampled device totals including context; shorter-lived peaks can be missed.

Assignment-product capacity uses an independent configured row count a_{assign}, selected by `JHQ_ASSIGN_BATCH` or the source default: 32,768 for at least 8,192 lists, otherwise 8,192.

$$S_{\text{dots}} = 4n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

Allocation capacity is independent of active/staged rows. At $d = 1536/3072$, defaults both allocate 3 GiB. For BGE/Stella, $3 \times 4 = 12$ GiB assignment scratch precedes gathering; figure 4.4 separates those lifetimes.

$$S_{\text{peak}} = \max_t \sum_{A \text{ live at } t} |A|.$$

Stella's smaller repack plus model/context gives its lower observed peak.

Table 4.2. Analytical row-array occupancy (GiB), $d = 1{,}024$, $M = 128$, $B_r = 8$. BGE uses one pass; Stella uses two. The one-pass Stella column is counterfactual capacity analysis. Pipeline, model, offsets and runtime costs are excluded from these phase values.

Dataset	Final row payload	One-pass gather	Two-pass repack
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

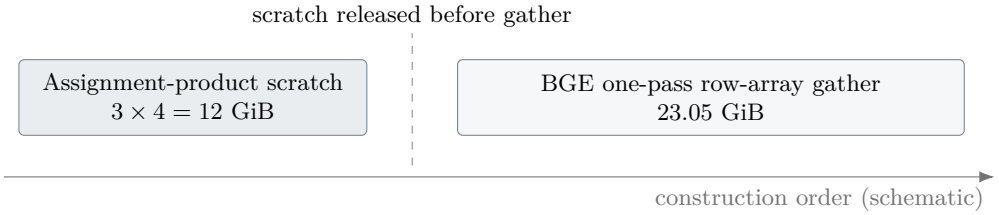


Figure 4.4. Selected construction lifetimes, not total process occupancy. BGE's 12 GiB assignment buffers are freed before its 23.05 GiB row-array gather. These separate terms must not be summed as concurrent memory.

4.6 Index Representation and Safety Contracts

A signed-32-bit $n_t d$ overflow at 2^{31} zeroed scale; a version-blind cache reloaded it. Chunked FP64 norms fixed the fault; routing success alone missed it.

4.6.1 Completed representation evidence

Pilot exports cover Vogue, arXiv and OpenAI-3072.

Codebooks passed separability, scalar-level and sorting checks. An FP64 loop checked $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$ on every exported row; maximum absolute error was 1.28×10^{-7}–2.77×10^{-7}. Sampled GPU-returned top-10 scores have maximum relative errors from 2.38×10^{-7} to 4.85×10^{-5}, using $|s_{\mathrm{GPU}} - s_{64}|/s_{64}$ with the query-norm constant included. These are checks of the exported representation, not re-encoding from the original input vectors.

Across seven workloads and 896 query instances, the pilot additionally checked the proposed residual bounds against FP64 reconstructed distances for 72,800,590 routed candidate instances, with zero reported violations. The same population supports the FP64 composite-score identity check. GPU-returned-score checks cover only top-10 slots and skip IDs absent from the routed map, leaving output membership incompletely checked.

Encoder, primary/residual training and construction sources match the evaluated implementation after namespace normalisation and removal of the export method. This source correspondence covers the shared path, leaving later model identity and large-dataset two-pass construction to separate checks.

4.6.2 Independent input and placement audit

The supplementary exporter audit uses the shared construction path on Vogue, arXiv, OpenAI-3072 and Stella. Offsets are monotone with endpoints 0 and N; IDs form a permutation of every database row in all four. FP64 products of the stored rotation give maximum entrywise $\|\Pi\Pi^{\mathsf{T}} - I\|$ errors of 3.96–4.34×10^{-7}. Sampled norm checks use FP64 rotation, rather than the GPU GEMM, and therefore measure the stored matrix's geometry. Table 4.3 summarises sampled recoding.

Table 4.3. Original-input FP64 recoding audit. Flips count coordinate decisions; the final column is the largest residual-boundary distance among mismatches. Structural checks cover all rows; recoding is sampled.

Dataset	Rows	Orth. error	Primary flips	Residual flips	Max margin
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

At 768/1,024 dimensions, recoding mismatches fall within 1.12×10^{-7} of boundaries. OpenAI-3072 has 2.68% residual mismatches, up to 2.67×10^{-5} from boundaries, consistent with TF32. Conditioning on stored primary digits avoids double-counting flips.

ID-joined builds have identical codes on three smaller databases, corrections within 8.94×10^{-8}, and 1/1/40 Vogue/arXiv/OpenAI assignment differences. Stella checks only two-pass construction. Missing exported centroids leave membership unverified.

4.6.3 Contract coverage and remaining checks

Table 4.4. Representation contracts after the pilot and supplementary input audit. Checks separate geometry, arithmetic and row placement.

Boundary	Required property	Evidence / remaining check
C0: geometry	Common transform with bounded floating-point isometry error	Stored-matrix orthogonality checked on four datasets; TF32 GPU arithmetic distinguished
C1: codebooks / correction	Cartesian levels, sorted residual codebooks, correction matches decoded codes	Pilot checks completed on three databases; all-row correction errors in section 4.6
C1: input encoding	Stored codes encode the intended original input row	FP64 original-input recoding on four samples; boundary and TF32 discrepancies quantified
C1: placement	Offsets cover rows; IDs form a permutation; all fields follow it	All-row offsets/ID checks; three ID-joined path comparisons; centroid membership unverified
Execution	Valid indexing, synchronisation and score-order contracts	Archived query and sanitizer checks; finite coverage, detailed in section 7.4

Required conditions are

$$M \mid d, \quad D_s \mid B, \quad M \bmod 4 = 0.$$

Require finite inputs/valid counts and sentinels. IDs are signed 32-bit; address products use wider arithmetic.

T4 validates selection (section 7.4.1); centroid membership, empty lists and adversarial tails remain uncertified.

5 Representation-Aware GPU Search

GPU scoring/selection uses JHQ's hierarchy [16]. Chapters 6 and 7 evaluate it and set budgets.

5.1 Query Path and Four Distinct Ranking Objects

Queries rotate, route, screen and refine to return k IDs/scores. Figure 5.1 distinguishes the three runtime selection decisions; the ground-truth ordering is an external evaluation object.

Four quantities must be kept separate:

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

μ_i is a transformed centroid; $\widehat{p}, \widehat{r}$ are decoded primary/residual vectors. The runtime refined score includes a query-only constant, as derived from the implemented correction in Section 5.6. It preserves the ordering of reconstructed distances for a fixed query, but neither it nor the primary score is the original-vector distance.

Routing defines the visited universe U of n_c candidate occurrences; primary screening retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ for residual scoring. Table 5.1 records each stage's inputs, reads and costs; sections 5.2 to 5.7 develop their implementations.

The ordering and exactness columns of table 5.1 attach each claim to its score and tie rule. Chapter 7 tests survivor sets, ordered outputs and raw-space recall separately.

Table 5.1 gives the stage contracts. The E1/budget harness explicitly selects 16 refinement lanes and scan policy -1 (scheduling among 0/2/4/8), versus library defaults of scalar refinement and policy 0. Shared bounds are enabled but used only at $g_{\mathrm{scan}} > 1$; the harness calibrates scheduling with 512 database pseudoqueries, a step the library does not perform automatically. Component overrides and implementation versions are in table 3.4.

5.2 Exact Cartesian LUT Factorisation

Primary ADC uses FP32 q' and stored reconstructions [16, 17]:

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\mathrm{P}}(q,x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

The 256-entry table can shrink because each subcodebook is Cartesian.

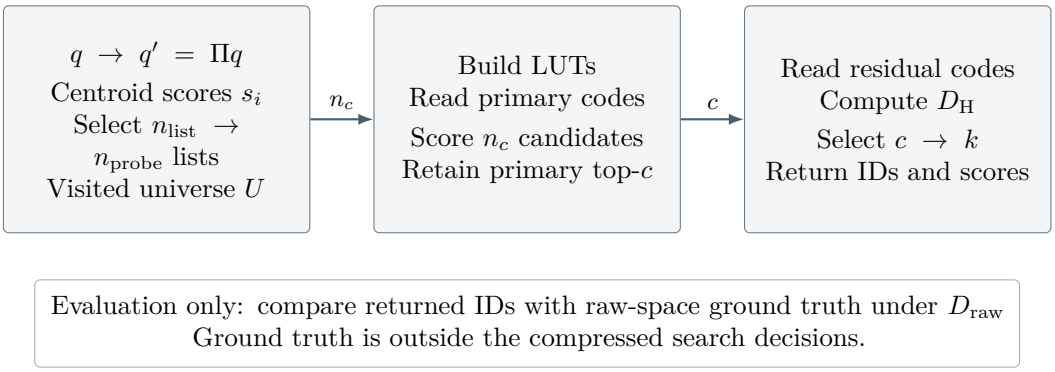


Figure 5.1. Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions.

Table 5.1. Query-stage contracts. $c_v = \min(c, n_c)$ counts valid primary survivors. Exactness refers to the stated computed scores; routing and quantised reconstruction determine raw-space approximation.

Stage	Input → output	Data read	Main work	Order / exactness
Transform	$q \to q'$	Query; transform	Apply Π	Transform (precision in section 7.2.3)
Probe	$n_{\text{list}} \to n_{\text{probe}}$ lists	FP32 centroids; list lengths	Score; radix; rank count	(s_i, i); exact probe selection
LUT	$q' \to$ tables	Primary codebook	$MD_s 2^{8/g_{\text{LUT}}}$ terms	Real-arithmetic factorisation; FP32 rounding
Primary, $g_{\text{scan}} = 0$	$n_c \to c_v$	Packed primary codes; LUT	$O(n_c M)$ scoring; compaction	Exact distance-order selection; arbitrary cutoff ties
Primary, $g_{\text{scan}} \geq 1$	$n_c \to c_v$	Same primary representation	Score; compact; merge	(D_{P}, p); exact under stated premises
Refine	$c_v \to c_v$ scores	Residual codes; correction	$O(c_v d)$	Computed D_{H}; lane-dependent FP32 association
Final	$c_v \to k$	Refined scores; IDs	$O(kc)$ slot scans	$(D_{\text{H}}, \text{ID})$; exact with k finite valid entries

For $K = 256$, $D_s = 8$, the analytical codebook gives

$$a = \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8,$$
$$D_{\text{P}}(q, x) = \|q'\|_2^2 + da^2 - 2a \sum_{j=0}^{d-1} s_j(x) q'_j. \tag{5.3}$$

Here Φ is the standard normal CDF; a byte stores eight signs, higher bits first. Thus

$$a > 0 \quad \Longrightarrow \quad \arg\min_x D_{\text{P}}(q, x) = \arg\max_x \sum_j s_j(x) q'_j.$$

Four-/eight-way splits retain the same real-arithmetic score. Learned replacements require compatibility checks.

5.2.1 Factorisation without changing the quantiser

Write $z = 16h + l$, with $h, l \in \{0, \ldots, 15\}$ and $\mathcal{C}_m[z] = (A_m[h], B_m[l])$. Then

$$T_m^{\text{hi}}[h] = \|q'_{m,\text{hi}} - A_m[h]\|_2^2, \qquad T_m^{\text{lo}}[l] = \|q'_{m,\text{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

Coordinate separation gives

$$T_m[z] = T_m^{\text{hi}}[z \gg 4] + T_m^{\text{lo}}[z \,\&\, 15]. \tag{5.5}$$

Storage falls to $16 + 16 = 32$ entries; codes, candidates and real-arithmetic scores stay fixed.

The builder slices codewords $16h, l$; `verify_split_lut` checks nibble/slice compatibility. An arbitrary learned 256-centroid PQ subcodebook need not satisfy this condition: dividing its code labels into nibbles does not factor its centroid coordinates.

The same-candidate FP32 audit in section 7.3 measures numerical and ranking differences between table forms, including changes after identical primary survivor sets.

5.2.2 Grouping, construction work and resource cost

The table-grouping ablation generalises the split to $g_{\text{LUT}} \in \{1, 2, 4, 8\}$ contiguous coordinate groups whose bit fields match the Cartesian digits. For eight bits and $D_s = 8$, each group has

$2^{8/g_{\mathrm{LUT}}}$ entries; the total entries and FP32 storage per query are

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

Independent entry construction evaluates $D_s 2^{8/g_{\mathrm{LUT}}}$ coordinate contributions per subspace. Two groups therefore reduce table storage eightfold and coordinate work sixteenfold relative to independently constructing every full-table entry. These are table-level cost ratios, not predicted application speedups.

The table is built in global workspace and staged cooperatively when its complete shared-memory footprint fits the queried per-block opt-in limit $S_{\max}$. For buffer capacity C_{buf}, ordinary/grouped scan footprints are

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

For the evaluated paths (without optional fused-refinement/early-exit scratch),

$$\text{shared lookup} \iff S_{\mathrm{path}} \le S_{\max}.$$

In the table sweep, $c = 1{,}000$, block sizes are 512 or 1,024, and $C_{\mathrm{buf}} = 2{,}048$. Full tables span 96–384 KiB, versus 12–48 KiB for two groups. NVIDIA specifies a 99 KiB per-block limit for compute capability 12.0 [24]. With this limit, the full-table footprint exceeds capacity even at $M = 96$ (114,696 bytes in the query-centric path), whereas the largest two-group footprint is 65,560 bytes in the grouped path. Thus the source-level placement rule sends full tables to global memory and two-group tables to shared memory in these configurations. This placement is inferred from the launch rule and published limit. The measurement records do not include a device-attribute query.

Four and eight groups both store 16 entries per subspace but perform more lookups and combinations. Extraction/dependencies/staging/synchronisation/residency set the trade-off. Each scan group may stage a copy.

The 16 + 16 nibble LUT follows small-table precedents [1, 2, 33]. Here GPU retention/refinement integrate it with JHQ's two-level score; measured trade-offs follow in section 7.3.

5.3 Packed Codes and Candidate-Parallel Memory Access

Subspace-major codes align neighbouring lanes/candidates; splitting reduces LUT storage.

5.3.1 Address mapping and four-byte packing

For list ℓ, row r maps to

$$p = o_\ell + r, \qquad \mathrm{offset}(Z[m,p]) = mN + p.$$

At fixed m, $\Delta\,\mathrm{offset} = \Delta p$. List boundaries can break adjacency.

Pack four subspace bytes per candidate:

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t,p], \qquad \mathrm{offset}_{\mathrm{bytes}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{5.8}$$

Table 5.2. Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts.

g_{LUT}	Entries	FP32 bytes	Lookups	Combining adds	Build terms
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

For $0 \le t < 4$ and eight-bit Z,

$$(\mathsf{w}[j,p] \gg 8t)\ \&\ 255 = Z[4j+t,p].$$

Nibbles index equation (5.5).

Construction maps $[M,N]$ bytes to $[M/4,N]$ words $(4 \mid M)$. Casting would combine candidate bytes. Code-byte costs:

$$R = W = NM, \qquad S_{\text{old+new}} = 2NM.$$

Repacking is outside search timing.

5.3.2 What packed access saves, and what it preserves

One word load/unpack replaces four byte loads. The logical warp window is $32 \times 4 = 128$ bytes; transactions depend on alignment/active lanes. Payload stays M bytes per candidate. Both arms are subspace-major, so transpose/cache effects are not isolated.

5.4 Exact Global Primary Top-c via Multi-Block Scanning

Groups scan in parallel and merge computed-primary survivors [18].

5.4.1 Candidate partition and deterministic ordering

For a fixed query, concatenate the selected inverted lists in probe order to obtain n_c candidate occurrences. Assume that each visited database row occurs once and that the index stays fixed during search. For $g_{\text{scan}} \in \{1,2,4,8\}$, block (q,j) owns the interval

$$U_j = \left[\left\lfloor \frac{jn_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1)n_c}{g_{\text{scan}}} \right\rfloor\right), \qquad j = 0,\dots,g_{\text{scan}}-1. \tag{5.9}$$

Here an interval denotes the corresponding candidates, rather than their database IDs. The device computes its endpoints from the routed candidate count. Groups differ in size by at most one candidate; a long list can cross a group boundary. This balances candidate counts, whereas assigning equal numbers of lists need not do so. Each thread finds its first list through the probe-prefix offsets and advances its list cursor while scanning its interval. Residual codes are not read at this stage.

The selection key is

$$K_q(x) = (D_{\mathrm{P}}(q,x), p(x)), \qquad (a,p) \prec (b,r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

where $p(x)$ is the physical row position in the fixed inverted-list storage. It is unique among the query's candidates and independent of the order of atomic appends. It is not the database

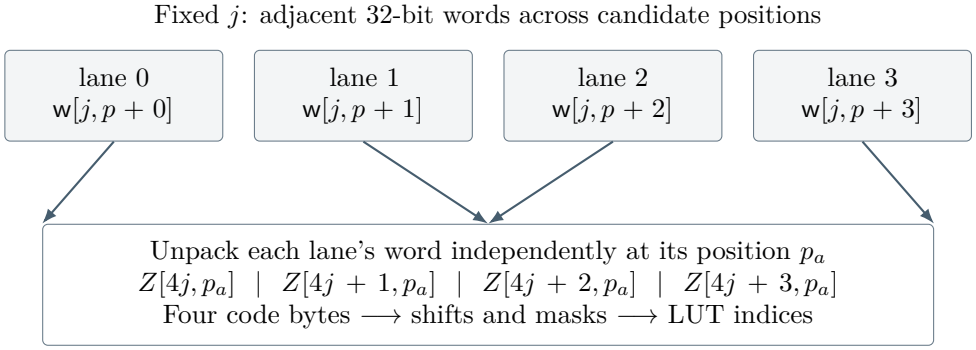


Figure 5.2. Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions.

ID used to break final-result ties. With non-NaN scores, this defines a total order on distinct candidates. Local selection and merging use this order. Partition independence also requires identical computed scores across paths.

5.4.2 Exact local retention and global coverage

$\mathrm{Top}_c(V)$ contains the first $\min(c, |V|)$ keys, where $c = \max\{k, \lceil \alpha k \rceil\}$. Each group keeps up to c, since all global winners may lie there; a c/g_{scan} quota is unsafe.

Lemma 5.1 (Local-buffer retention)**.**
Let V_t be the candidates scored by a block through chunk t, and B_t its valid buffer after any compaction or filtering at that point. Assume fixed complete keys, sufficient append capacity and exact top-c compaction. Let β_t be the minimum of the complete shared bounds observed so far, including the initial top sentinel. Then

$$\mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

With sharing disabled, the bound is the sentinel and the buffer contains the whole prefix top-c.

Proof.
The empty prefix is immediate. Put

$$E_t = \mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\}.$$

On extending V_t to V_{t+1}, $\beta_{t+1} \preceq \beta_t$ and

$$\mathrm{Top}_c(V_{t+1}) \cap V_t \subseteq \mathrm{Top}_c(V_t).$$

Consequently,

$$x \in E_{t+1} \cap V_t \Rightarrow x \in E_t \subseteq B_t.$$

A new key rejected locally has c predecessors, hence rank $> c$. Capacity admits the remaining eligible arrivals. For any eligible winner x,

$$\#\{y \in B : K_q(y) \prec K_q(x)\} \leq \#\{y \in V_{t+1} : K_q(y) \prec K_q(x)\} < c,$$

so exact compaction keeps x. Filtering removes $K > \beta_{\mathrm{seen}}$; the running minimum covers stale reads. These steps preserve the invariant inductively. □

Proposition 5.1 (Coverage and truncated merging)**.**
For fixed keys under a common total order, let U be the visited universe and V any retained subset. If $\mathrm{Top}_c(U) \subseteq V \subseteq U$, then $\mathrm{Top}_c(V) = \mathrm{Top}_c(U)$. Pairwise sorted merging with truncation to c preserves this result. In particular, for disjoint groups,

$$\mathrm{Top}_c\left(\bigcup_j U_j\right) = \mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

The statement includes universes smaller than c.

Proof.
Every global winner is in V, and every other member of V follows the global winners. Selection from V therefore recovers them. For the independent-group special case, let x belong to the global top-c and to group U_j. Fewer than c candidates precede x globally, so fewer than c precede it within U_j. Consequently $x \in \mathrm{Top}_c(U_j)$, and the local union contains every global winner. Selecting its first c therefore recovers precisely the global winners in the same order. If the whole universe has fewer than c members, every member is retained.

For merging, the first c of two sorted lists are their union's top-c. A discarded candidate already has at least c predecessors within that union and cannot enter the top-c of any larger union. Induction over the merge tree establishes the claim. □

The CUDA implementation maintains a shared-memory buffer of C_{buf} key pairs, where C_{buf} is a power of two and $C_{\mathrm{buf}} \geq c + T_{\mathrm{blk}}$ for a scan block of T_{blk} threads. Bitonic compaction keeps up to c valid keys and sets the threshold to the last key once c exist. Before that point the threshold is a sentinel above every valid key. Between compactions, admission requires $K \prec \lambda_j$. The threshold has c witnesses, so remains an upper bound on the prefix cth key as candidates arrive.

Capacity follows from

$$|B| + |\Delta V| \leq c + T_{\mathrm{blk}} \leq C_{\mathrm{buf}}.$$

This supplies lemma 5.1's append premise. With propositions 5.1 and 5.2, filtering/merging retain global winners.

Explicit scores need $O(n_c)$ writes/storage. Fused buffering emits $O(g_{\mathrm{scan}}c)$ pairs; register selection is another option [18].

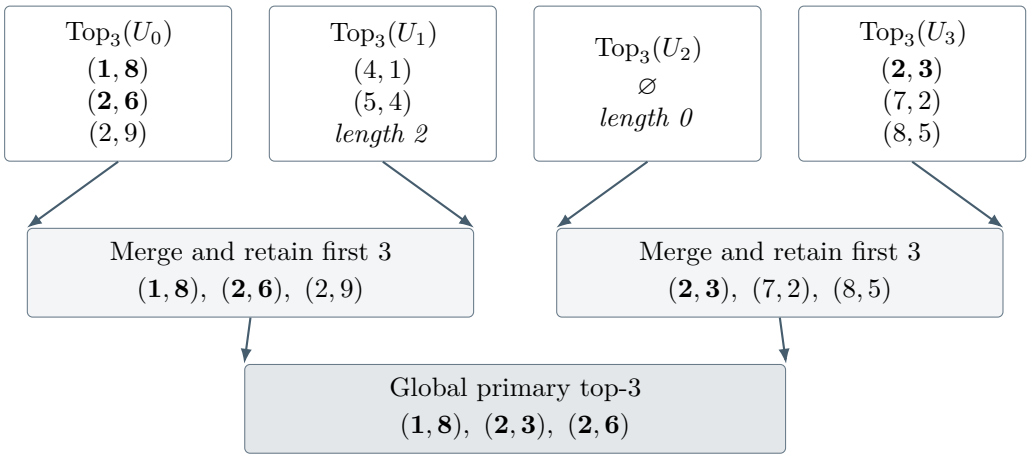


Figure 5.3. Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $g_{\mathrm{scan}} = 4$. Entries are illustrative (D_{P}, p) keys; bold ones are global winners. Short/empty groups contribute their valid prefixes. Merges keep at most c in the same order.

5.4.3 Merge implementation, storage and boundary cases

At $g_{\text{scan}} > 1$, a 256-thread block merges each query over $\log_2 g_{\text{scan}}$ rounds. Each pair emits $\min(c, a + b)$ entries using Merge Path slices [14]. All comparisons use equation (5.10); block barriers separate rounds. Figure 5.3 illustrates the truncation invariant. The tree emits at most $c(g_{\text{scan}} - 1)$ key pairs across its internal nodes, in addition to the partition searches and synchronisation work.

Two alternating pair workspaces have shapes $[b_{\text{cap}}, g_{\text{scan}}, c]$ and $[b_{\text{cap}}, g_{\text{scan}}/2, c]$:

$$S_{\text{pairs}} = 8 b_{\text{cap}} c \left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2} \right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

Counts/bounds add $4 b_{\text{cap}} g_{\text{scan}}$ and, when provisioned, $8 b_{\text{cap}}$ bytes; survivor/other workspace is additional. Allocations persist after growth before capture. Merge shared memory holds lengths, not all keys.

Counts delimit valid prefixes:

$$K_{\text{sort pad}} = (+\infty, \text{INT_MAX}), \qquad K_{\text{output pad}} = (+\infty, -1).$$

Valid positions are below INT_MAX; empty groups have zero count. Negative positions are skipped/excluded. $g_{\text{scan}} = 1$ writes directly.

One block ($g_{\text{scan}} = 0$) is distance-exact with free cutoff ties. Lossy `K_LOCAL=4` requires `JHQ_EXACT_TOPCK=0`, disabled in paper-faithful builds.

For nonempty V, put $h = \min(c, |V|)$, with hth score d_h. Distance-only compaction yields

$$\{x : d_x < d_h\} \subseteq B \subseteq \{x : d_x \leq d_h\}, \qquad |B| = h.$$

Each rejection has c no-larger witnesses; tied IDs remain free. T4 checks this contract and grouped full-key order by CPU sorting GPU-produced scores (section 7.4.1).

5.5 Shared Complete-Key Thresholds and Execution Scheduling

Local top-c can emit $g_{\text{scan}} c$ intermediate entries for c winners. Shared bounds reduce this maintenance. The scheduler then chooses how much scan parallelism to expose. Both operate at a fixed refinement budget; choosing that budget is the separate problem in Chapter 6.

5.5.1 A complete-key upper bound with explicit witnesses

Let $K_{(c)}$ be the global cth key when $|U| \geq c$. If a group holds c distinct, fully scored candidates, the largest key among those witnesses, τ_j, satisfies $K_{(c)} \preceq \tau_j$. There are already c candidates no later than τ_j in the global universe. The shared bound is initialised to a top sentinel and updated by

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

No bound is published from fewer than c valid candidates. If the entire universe has fewer than c, no group can publish such a bound and all valid candidates remain eligible.

Proposition 5.2 (Safe rejection by a witnessed bound)**.**
Assume fixed keys, publications backed by c distinct candidates, and reads that return either the initial sentinel or one complete published key. Rejecting a candidate only when $K_q(x) \succ \tau$ cannot remove a global top-c candidate, irrespective of publication timing.

Proof.
A publication has a set W_τ of c distinct witnesses:

$$\max_{x \in W_\tau} K_q(x) \preceq \tau \quad \Longrightarrow \quad K_{(c)} \preceq \tau.$$

Sentinel reads and minima of published keys obey the same inequality. Thus

$$x \in \mathrm{Top}_c(U) \implies K_q(x) \preceq K_{(c)} \preceq \tau_{\mathrm{seen}} \implies x \text{ is not rejected.}$$

Stale reads only loosen the bound. Publications made after earlier safe rejections remain valid: any c retained distinct candidates still supply c witnesses, whether or not they are the best c of their group. □

Shared filtering retains global coverage (proposition 5.1). Admission is

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\mathrm{seen}}, \tag{5.15}$$

Here λ_j is the local compaction threshold and τ_{seen} the last shared key read. The shared comparison retains equality.

5.5.2 Publication, filtering and implementation limits

For nonnegative FP32 scores, canonical positive zero and valid nonnegative positions,

$$\kappa(D, p) = 2^{32}\,\mathrm{uintbits}_{32}(D) + p. \tag{5.16}$$

On this non-NaN domain, monotone float bits give

$$(D_1, p_1) \prec (D_2, p_2) \iff \kappa(D_1, p_1) < \kappa(D_2, p_2).$$

Finite keys precede infinity and the all-one sentinel. Invalid positions cannot publish.

A thread polls each chunk. After compaction, c valid keys permit 64-bit `atomicMin`. Final publication/search trims the prefix; exact merge handles stale extras. In-stream reset precedes scanning, then merging; groups never wait.

The whole-key premise uses aligned eight-byte strong PTX `ld.volatile.global.u64`/`atom.global.min.u64` [26], beyond C++ `volatile` [25]. CUDA 13.0.88 `ptxas`, `sm_120` corroborates it; driver-JIT SASS was not captured.

Filtering follows full primary scoring (figure 5.4): maintenance savings compete with polling/atomics (section 7.4).

5.5.3 Choosing parallelism without changing the budget

bg_{scan} blocks add parallelism and staging/selection/merge costs. Smaller quotas require a certificate or fallback. Concentrated pilot survivors motivated witnessed bounds (section 7.4).

Define

$$T_{\mathrm{blk}} = \text{scan threads/block}, \quad H = n_{\mathrm{SM}} T_{\mathrm{resident/SM}}, \quad v = \frac{\mathrm{sd}(n_c)}{\mathrm{mean}(n_c)}.$$

Its rule is

$$g_{\mathrm{fill}} = \max(\{1\} \cup \{h \in \{2, 4, 8\} : bhT_{\mathrm{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\mathrm{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\mathrm{chosen}} = \max(g_{\mathrm{fill}}, g_{\mathrm{skew}}). \tag{5.18}$$

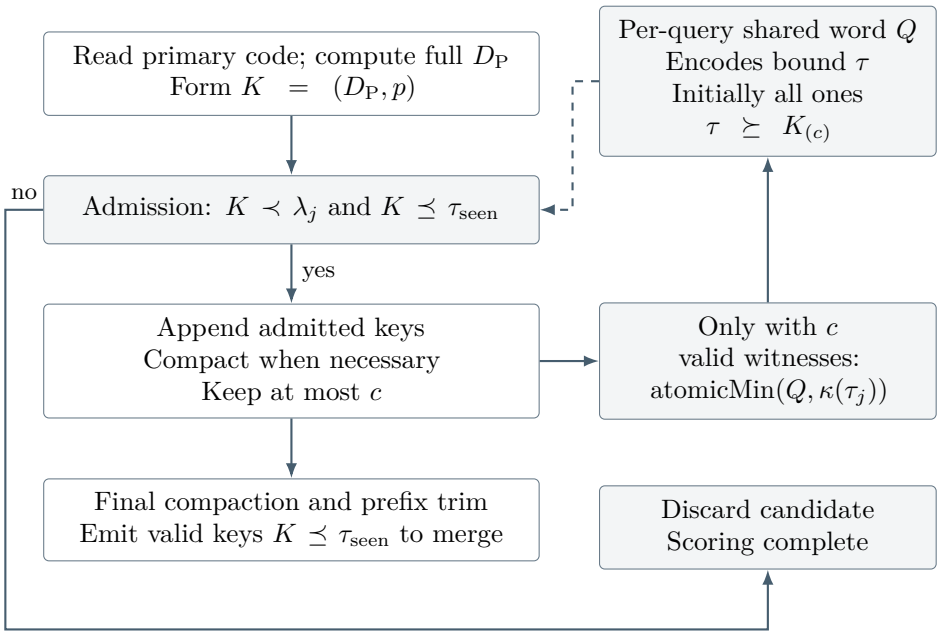


Figure 5.4. Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text.

The skew rule may override the fill limit. In the evaluation harness, v is calibrated using 512 database vectors as pseudoqueries and a fixed routing configuration; it uses no ground-truth labels. This is distinct from Chapter 6's sampled output-agreement calibration. Once calibrated, scheduling uses the batch size and stored statistic without an additional per-search host readback. The constants were fitted on four pilot configurations and then evaluated on further configurations. The one-time pseudoquery calibration is excluded from the reported steady-state search times. Observed startup ranges, separate from refinement-budget calibration (section 3.5.3), are

$$T_{\text{gather}} \in [0.81, 7.04]\,\text{ms}, \quad T_{\text{first}} \in [6.00, 27.77]\,\text{ms}, \quad \text{median}(T_{\text{repeat}}) \in [2.72, 18.13]\,\text{ms}.$$

Choosing one group dispatches $g_{\text{scan}} = 0$ at fixed index/probes/budget. Its distance-only ties need a common-order check for identical survivors. Integrated gains/regret are in section 7.6.

5.5.4 Observed execution paths

On the 170-SM RTX 5090 (48 warps/SM) [24],

$$1000 \cdot 2 \cdot 512 = 1{,}024{,}000 > 0.6 \cdot 170 \cdot 48 \cdot 32 = 156{,}672.$$

Fill therefore chooses one group; skew can override to four.

In E1, 9/38 records cross $v = 0.3$, all Vogue (0.338–0.622). Its 27 cells use four groups; 105 arXiv/OpenAI cells use $g_{\text{scan}} = 0$. Their gains use LUTs, packing, lanes and probe selection, without grouped retention/bounds.

Vogue: 3–15 pure-tie differences/1,000 queries, zero non-tie differences, identical final outputs. At 128 probes the tie range is 7–11.

Observed cell counts are

	$G = 0$	$G = 2$	$G = 4$
held-out	10	0	2
$S = 128$	2	8	2

Eight of twelve switch paths. Benefits depend on load size/skew, without a fixed batch-256 cutoff.

The heuristic omits residency/compaction/merge costs; pseudoqueries can age. Worst measured regret is 1.217 at batch 256. Budgets stay fixed; section 5.6 handles refinement.

5.6 Lane-Cooperative Residual Scoring and Final Top-k

Primary screening changes coordinate work from $O(n_c d)$ to $O(cd)$. Lanes cooperate at the same budget.

5.6.1 Reconstructed scoring and the query-only constant

For $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$, expand in real arithmetic:

$$D_{\mathrm{H}}(q,x) = D_{\mathrm{P}}(q,x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.19}$$

$$= 2\|q'\|_2^2 + \|\widehat{p} + \widehat{r}\|_2^2 - 2\langle q', \widehat{p} + \widehat{r} \rangle$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.20}$$

The query constant preserves reconstructed-distance order [16].

Scalar scoring sums squared residual differences, then adds primary score/correction. Queries and fitting codebooks use shared memory.

5.6.2 Mapping lanes to words and reducing partial sums

For $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$,

$$d_{\mathrm{word}} = 32/B_R, \qquad \text{words of lane } a: \ a, a+L, a+2L, \ldots.$$

A word holds four eight-bit or eight four-bit codes. Butterfly shuffles sum lane partials; lane zero writes the score.

Word-aligned rows and $d_{\mathrm{word}} \mid D_s$ keep words within codebooks. All lanes shuffle; invalid slots contribute zero, then output infinity. $L = 16$ gives two candidates/warp.

L0 loads bytes; L1 loads words with the same one-lane accumulation. L8/L16/L32 trade shorter walks for shuffles/fewer concurrent candidates. FP32 association and locality matter.

5.6.3 Final selection and the numerical boundary

For at least k finite valid survivors with unique IDs, each round computes

$$i^* = \arg\min_{i<c}(D_{H,i}, \mathrm{ID}_i), \quad d^* = D_{H,i^*}, \quad D_{H,i^*} \leftarrow +\infty$$

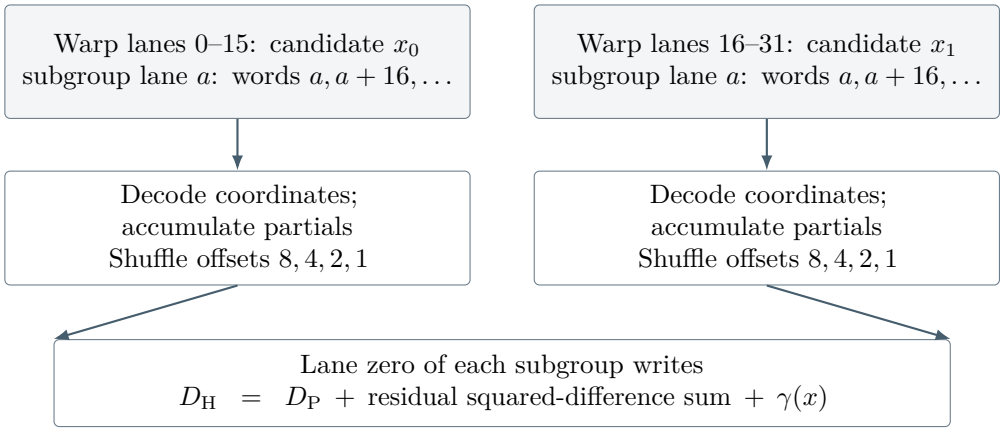


Figure 5.5. Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a+16, \ldots$. Each eight-bit-code word decodes four coordinates. Subgroups sum partials, then add primary score/correction. Candidate positions may remain scattered.

using all slots and emits (ID_{i^*}, d^*): exact retained-pool top-k, without thread quotas. Underfull pools may repeat the smallest valid ID at infinity; all-invalid pools return invalid IDs. Tests meet the finite-entry premise.

Primary ties use positions, final ties IDs. Exclusions/quantisation remain; lane FP32 association may move near ties.

5.7 IVF Probe Selection as an Independent Primitive

Probe selection precedes primary scanning and selects inverted lists, not database-vector survivors. For IVF centroid μ_i, the selector orders the FP32-valued computed routing scores

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

Omitting the common query norm preserves centroid ordering. In contrast to primary squared-distance tables, these scores can be negative. The output is

$$n_{\mathrm{sel}} = \min(n_{\mathrm{probe}}, n_{\mathrm{list}}), \qquad I = \mathrm{Top}_{n_{\mathrm{sel}}}^{(s_i,i)}\{0, \ldots, n_{\mathrm{list}} - 1\},$$

with list-length prefix offsets.

OpenAI v69 uses TF32 rotation/centroid GEMMs with FP32 accumulation. Probe exactness concerns their computed scores; FP32 `fma` retains earlier rounding (section 7.2.3).

5.7.1 Radix selection followed by deterministic ordering

One block makes four MSB-first radix passes. For prefix-restricted histogram H_t and one-based residual rank r_t ($r_0 = n_{\mathrm{sel}}$),

$$b_t = \min\Big\{b : \sum_{a \le b} H_t[a] \ge r_t\Big\}, \qquad r_{t+1} = r_t - \sum_{a < b_t} H_t[a].$$

The chosen bin extends the prefix. Four bytes fix the cutoff/equal-key count.

Canonicalise signed zeros to positive zero, then map bits u by

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

For non-NaN scores, monotonicity follows by cases:

$$\begin{aligned} &D_1 < D_2 < 0 \Rightarrow u_1 > u_2 \Rightarrow \sim u_1 < \sim u_2, \\ &0 \le D_1 < D_2 \Rightarrow u_1 < u_2 \Rightarrow (u_1 \mid 2^{31}) < (u_2 \mid 2^{31}), \\ &D_1 < 0 \le D_2 \Rightarrow F(u_1) < 2^{31} \le F(u_2). \end{aligned}$$

Infinities/zeros are covered, unlike the nonnegative-only equation (5.16). Repeated `fma` fixes recomputed keys.

Histogram storage is

$$256 \text{ bins} \times 32 \text{ lane replicas} \times 4 \text{ bytes} = 32\,\mathrm{KiB}.$$

Replication reduces same-warp collisions, leaving inter-warp contention/reduction cost.

Let h be the cutoff key. Collection forms

$$A_< = \{i : F(u_i) < h\}, \quad A_= = \{i : F(u_i) = h\}, \quad m = n_{\mathrm{sel}} - |A_<|.$$

Take $A_<$ plus the m smallest IDs in $A_=$, using ordered ballots/prefix counts rather than atomic arrival. For the resulting set A,

$$\mathrm{rank}(i) = |\{j \in A : (s_j, j) \prec (s_i, i)\}|, \qquad \mathrm{out}[\mathrm{rank}(i)] = i.$$

Selected lengths produce prefix offsets. For excess probes,

$$i_j = i_{n_{\mathrm{sel}}-1}, \quad L_j = 0 \quad (j \ge n_{\mathrm{sel}}), \qquad \omega_{j+1} = \omega_j + L_j.$$

Zero lengths prevent rescans. Require $n_{\text{list}}, n_{\text{probe}} > 0$.

5.7.2 Costs and separate correctness contracts

Histogram/collection reread scores; ranking is $O(n_{\text{sel}}^2)$ with growing shared memory. Small selections still pay fixed histogram storage; this is deterministic integration, not uniform linear cost.

Tests cover ties/zeros/infinities/extremes/excess probes (section 7.4).

FP16 approximation and cross-process checks are separate (section 7.4).